

AIML428 - NLP, LLMs Assignment #2

Jason Pollock
pollocjaso@myvuw.ac.nz, jason@pollock.ca

May 15, 2026

1 Task 1

1.1 Bar chart

I added two parameters to GPT.generate, image_filename and show_probs.

If show_probs is true, the top-10 indices are retrieved from the probs vector. Then the function plot_probs is called.

plot_probs decodes the indices to bytes for X, and creates a default colour vector. Then it finds the values in indices that have the chosen idx using pytorch, extracting the nonzero element from that tensor. It sets that colour to red.

It then plots the graph to <image_filename>_[0-9].png.

Given the prompt "I live in" it generated: " a diverse community". In the images below, we can see the second token was not in the top-10.

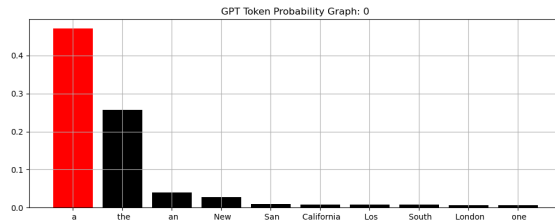


Figure 1: First token

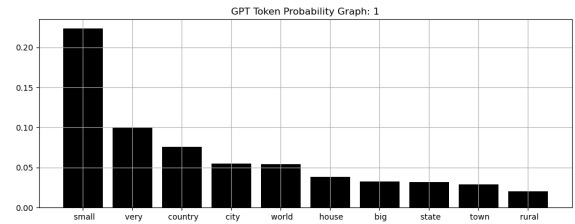


Figure 2: Second token

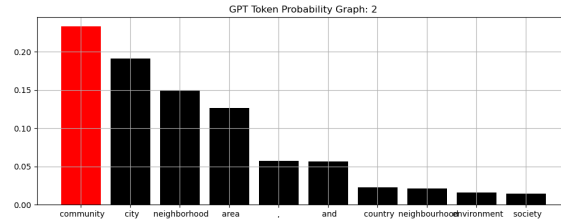


Figure 3: Third token

1.2 Temperature

Temperature affects the logits before softmax, making them smaller. This is then passed to softmax, which uses e^{Z_i} as the numerator. Making the exponent smaller will make the numerator smaller.

However, the effect is not equal to all values. If temperature is small, larger logits will grow, causing the numerator to grow exponentially.

For example:

$$temperature = 0.1 \quad (1)$$

$$Z_1 = 1 \quad (2)$$

$$Z_2 = 0.5 \quad (3)$$

$$Z_{t1} = 1/0.1 = 10 \quad (4)$$

$$Z_{t2} = 0.5/0.1 = 5 \quad (5)$$

As we can see, Z_1 is increased by 9, while Z_2 is only increased by 4.5. This then flows through to the exponent, making Z_1 240x more likely than Z_2 instead of the previous 10x.

Temperatures <1 shift the distribution towards the dominant value, while those >1 flatten the distribution, allowing tail values to be chosen.

For example, prompting with "I live in", with different temperatures produces:

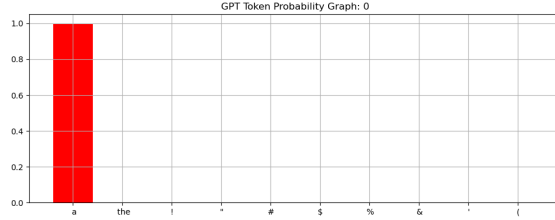


Figure 4: T=0.01

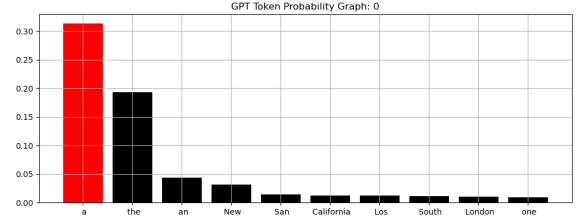


Figure 5: T=1.0

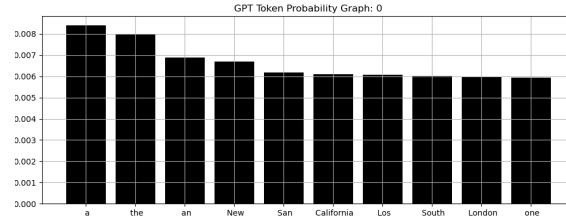


Figure 6: T=10.0

1.3 Response Probability

GPT.generate was modified to extract the selected token's probability and convert it to $\ln$ space. It was then added to an accumulator, eventually being returned as a tuple with the idx tensor.

Prompting with "I live in", and temperature = 0.0001, it generates "I live in a small town in the" with a total probability = 1.0. This makes sense

since the temperature will shrink the distribution down to 1 value, which is always chosen.

1.4 Probability Assumptions

The code currently assumes that there is only a single vector in the tensor being sampled.

It is important to use log space for these computations because multiplying small numbers makes them really small in a hurry. The floating point standard allows a CPU to become less accurate as the mantissa gets smaller. Remaining in log space allows sum to be used, staying away from the CPU's inaccuracy.

1.5 Fixed Response

sample.py converts the fixed_response string to a list of tokens. This is passed to GPT.generate. GPT.generate runs as before until just after it samples idx_next from probs. At that point, idx_next is popped off the front of fixed_response_ids. Then the code from before runs unchanged, calculating the log probability.

Prompting with "I live in" has the following results:

Fixed Response	Probability
" the"	0.257346
" the capital"	0.000483
" the capital of"	0.000115
" the capital of New Zealand"	3.22e-07

Table 1: 1.5 t-SNE Hyperparameters.

As the fixed string's length increases, the likelihood it was generated decreases.

2 Task 2

2.1 eval.py

I created an eval.py with an "eval" method in it. There doesn't seem to be a lot of value in having a method called eval which contains all of the code in eval.py. The code is a copy of sample.py, with the json file loaded and then iterating over the list within the ctx context. It ensures

the response starts with a " ". Without start/stop tokens, GPT2 requires proper whitespace between the request and response.

The eval function is the only code run once the model is loaded. It is the same code as sample.py, only retrieving the equivalent to "start" and "fixed_response" from the json file. I have added additional features:

- the ability to ignore the response, while using it to control the `max_new_tokens`
- support for start/stop tokens provided by the training pass
- ability to detect and report proper calculations
- ability to detect and report tool use

Samples of the request/response pairs:

```
[
{
  "prompt": "Janet\u2019s ducks lay 16 eggs per day. \nShe eats three for breakfast every morning \nand bakes muffins for her friends every day \nwith four. She sells the remainder at the \nfarmers ' market daily for $2 per fresh duck \negg. How much in dollars does she make every \nday at the farmers ' market?",
  "response": "Janet sells 16--3--4==<<16-3-4=9>>9 \nduck eggs a day. \nShe makes 9*-2=-$ \n<<9*2=18>>18 every day at the farmer\u2019s \nmarket. \n",
  "expected_value": "18"
},
{
  "prompt": "A robe takes 2 bolts of blue fiber and \nhalf that much white fiber. How many bolts \nin total does it take?",
  "response": "It takes 2\2=<<2\2=1>>1 bolt of \nwhite fiber \nSo the total amount of fabric \nis 2+1=<<2+1=3>>3 bolts of fabric \n",
  "expected_value": "3"
},
]
```

```

    "prompt": "Josh decides to try flipping a house. - -
    ↪ He buys a house for $80,000 and then puts in
    ↪ $50,000 in repairs. - - This increased the
    ↪ value of the house by 150%. - How much profit
    ↪ did he make?" ,
    "response": "The cost of the house and repairs came
    ↪ out to  $80,000 + 50,000 = \$$ 
    ↪  $\langle 80000 + 50000 = 130000 \rangle > 130,000$  \n He increased
    ↪ the value of the house by
    ↪  $80,000 * 1.5 = \langle 80000 * 1.5 = 120000 \rangle > 120,000$  \n So
    ↪ the new value of the house is
    ↪  $120,000 + 80,000 = \$$ 
    ↪  $\langle 120000 + 80000 = 200000 \rangle > 200,000$  \n So he made a
    ↪ profit of  $200,000 - 130,000 = \$$ 
    ↪  $\langle 200000 - 130000 = 70000 \rangle > 70,000$  \n" ,
    "expected_value": "70000"
  }
]

```

2.2 Dataset

I chose **GSM8K**. I wanted to explore how to improve the mathematical ability of a basic GPT. It contains request/response pairs representing Grade 8 (13 year old children) math problems.

Conversion is done using "make task5_dataset" which invokes "convert.py". Convert.py takes the parquet files placed in the "data" directory and converts them to json, splitting the training data into train and validation sets. The test set is written to the main directory.

2.3 Finetuning

I finetuned for 200 iterations, that was a good time/value tradeoff. Longer runs will probably improve results.

eval.py reports the log probability for each response, which is the basic check, but only really useful when looking at individual responses. It also reports the average perplexity over the total evaluation set. Average perplexity should allow us to more rapidly compare model passes, and is easy to calculate from the log probabilities. Since my dataset is math problems, I also check if the expected answer is contained in the response, and whether or not tool call tokens are also present.

These values only make sense when the response is unfixed. To get non-0 probabilities, the top-k value had to be increased to 50k. Otherwise, the fixed response probability was always 0.

I run the evaluation 4 times, fixed response, unfixed response on both the original and finetuned models.

The results were (INV indicates invalid due to fixed response):

Model Run	Avg Perplexity	P(Correct)	P(Tokens)
gpt2-large, fixed	4.49	INV	INV
gpt2-large, unfixed	3.39	0.2	0.0
gsm8k, fixed	2.383	INV	INV
gsm8k, unfixed	1.59	0.1	1.0

Table 2: 1.5 t-SNE Hyperparameters.

The perplexity of the responses dropped, indicating they were more likely to be produced post-fine tuning. However, the model produced more correct answers before fine tuning. The fine tuned model did consistently produce the tool call tokens "<<" and ">>".

I was surprised that the model learned a start token 'response:', a stop token '\n,' in addition to the tool call tokens.

Finetuning reduced perplexity substantially, reducing it by about 50% in all tests. Perplexity is the geometric mean of the token probabilities. It indicates the models average performance, and how likely it is to generate the entire corpus.

Two additional evaluation methods would use a judge. The experiment would A/B test the unfixed responses from both models, with the judge choosing between them. One uses a LLM as a judge, the other a set of humans. This would use ELO scoring, choosing between several possible responses. These would then be fed back to the model for additional fine tuning.

LLM judging is reinforcement learning. Human judging is reinforcement learning from human feedback.

3 Task 3

3.1 Exploring PyTorch and Train.py

There was a dramatic difference in performance between train.py and eval.py. train.py was able to perform 200 passes in around 90 minutes. A

test pass using 20% of the data would have taken 10+ hours. This is an investigation of how batch processing could be added to GPT.generate.

Going through the data handling in train.py, it is loading the tokens and instantly converting each token to 64 bits. This is done to avoid integer overflows when calculating intermediate weights. While the int16 might be sufficient, it doesn't leave much headroom with 50k tokens.

NanoGPT uses torch.stack to setup block processing. However, torch.stack assumes that all tensors have the same length. If the requests or responses are of variable length, they cannot be stacked and cannot be batched. It will be possible to perform beam search using this - the beams all grow at the same rate and have the same prefix. However batch processing of multiple different prompts doesn't match stack's requirements.

We can achieve this by padding the prompts and using a custom attention mask, but this isn't currently implemented in nanoGPT.

Another method would be to start with the tensors at the shortest common string, and then use the fixed_response logic to grow the pruned prompts until they are all of the same length and able to grow together. However that wouldn't do proper probability calculations, and it would either generate too many or not enough tokens.

A better algorithm would be:

```
incoming_idx[] — list of tensors for each
    ↪ prompt
set idx to the tensors in incoming_idx with
    ↪ the min length.
begin generation loop
    as idx grows, add in longer prompts
    track each idx's generated length, remove
        ↪ when max_tokens reached, or on
        ↪ stop_token
when idx is empty, return the completed idx
    ↪ tensors
```

That variant would calculate the accumulated log probability properly as well.

The PyTorch Module's use of `__call__` makes it hard to track through the code. Generally IDE's like PyCharm help a lot in moving through the codebase. It was interesting to see "design for extension" being used in `__call_impl` through the hook interfaces.

3.2 Extending NanoGPT

The extension is Beam Search. It will allow the model to search through the token space, possibly avoiding high probability starting tokens that result in low probability full responses.

This is implemented by wrapping the idx handling logic in GPT.generate.

```
while there are more tokens to generate
    current_beams ← beams to expand, initially
    ↪ start
    foreach beam in current_beams
        sample beam_width tokens without
        ↪ replacement
        append beam_width new beams to
        ↪ expanded_beams
    done
    sort expanded_beams
    replace current_beams with beam_width best
    ↪ beams
    from expanded_beams
done
return the idx and probability from the best
    ↪ beam in current_beams
```

For example, prompting "I live in" generates " a diverse community" with a total probability of 6.28e-5. However, using a beam_width of 5 and max_new_tokens = 3, we get " a small town" with a much higher probability of 8.1e-2, 1000 times more likely than the original. If we increase beam_width to 100, it doesn't find a better option.

The result should be compared to lowering the temperature on the prompt. There are extensions that rank beams based on edit distance as well as total probability, with a preference for diverse beams.

4 Appendix - AI Usage

I did not use any AI generated code.